Lesson 8: Array basics

Arrays are useful critters because they can be used in many ways. For example, a tic-tac-toe board can be held in an array. Arrays are essentially a way to store many values under the same name. You can make an array out of any data-type including structures and classes.

Think about arrays like this:

```
[][][][][][]
```

Each of the bracket pairs is a slot(element) in the array, and you can put information into each one of them. It is almost like having a group of variables side by side. Lets look at the syntax for declaring an array.

```
int examplearray[100]; // This declares an array
```

This would make an integer array with 100 slots, or places to store values(also called elements). To access a specific part element of the array, you merely put the array name and, in brackets, an index number. This corresponds to a specific element of the array. The one trick is that the first index number, and thus the first element, is zero, and the last is the number of elements minus one. 0-99 in a 100 element array, for example.

What can you do with this simple knowledge? Lets say you want to store a string, because C++ has no built-in datatype for strings, you can make an array of characters.

For example:

```
char astring[100];
```

will allow you to declare a char array of 100 elements, or slots. Then you can receive input into it it from the user, and if the user types in a long string, it will go in the array. The neat thing is that it is very easy to work with strings in this way, and there is even a header file called cstring. There is another lesson on the uses of strings, so its not necessary to discuss here.

The most useful aspect of arrays is multidimensional arrays. How I think about multi-dimensional arrays:

```
[][][][][]
[][][][][]
[][][][][]
[][][][][]
[][][][][]
```

This is a graphic of what a two-dimensional array looks like when I visualize it.

For example:

```
int twodimensionalarray[8][8];
```

declares an array that has two dimensions. Think of it as a chessboard. You can easily use this to store information about some kind of game or to write something like tic-tac-toe. To access it, all you need are two variables, one that goes in the first slot and one that goes in the second slot. You can even make a three dimensional array, though you probably won't need to. In fact, you could make a four-hundred dimensional array. It would be confusing to visualize, however. Arrays are treated like any other variable in most ways. You can modify one value in it by putting:

```
arrayname[arrayindexnumber] = whatever;
```

or, for two dimensional arrays

```
arrayname[arrayindexnumber1][arrayindexnumber2] = whatever;
```

However, you should never attempt to write data past the last element of the array, such as when you have a 10 element array, and you try to write to the [10] element. The memory for the array that was allocated for it will only be ten locations in memory, but the next location could be anything, which could crash your computer.

You will find lots of useful things to do with arrays, from storing information about certain things under one name, to making games like tic-tac-toe. One suggestion I have is to use for loops when access arrays.

```
#include <iostream>

using namespace std;

int main()
{
  int x;
  int y;
  int array[8][8]; // Declares an array like a chessboard

  for ( x = 0; x < 8; x++ ) {
    for ( y = 0; y < 8; y++ )
      array[x][y] = x * y; // Set each element to a value
  }
  cout<<"Array Indices:\n";
  for ( x = 0; x < 8;x++ ) {
    for ( y = 0; y < 8; y++ )
      cout<<"["<<x<<"]["<<y<<"]="<< array[x][y] <<" ";
    cout<<"\n";
  }
  cin.get();
}
```

Here you see that the loops work well because they increment the variable for you, and you only need to increment by one. Its the easiest loop to read, and you access the entire array.

One thing that arrays don't require that other variables do, is a reference operator when you want to have a pointer to the string. For example:

```
char *ptr;
char str[40];
ptr = str;  // Gives the memory address without a reference operator(&)
```

As opposed to

```
int *ptr;
int num;
ptr = &num; // Requires & to give the memory address to the ptr
```

The reason for this is that when an array name is used as an expression, it refers to a pointer to the first element, not the entire array. This rule causes a great deal of confusion, for more information please see our Frequently Asked Questions.